

Unity 祝福系统拆解案

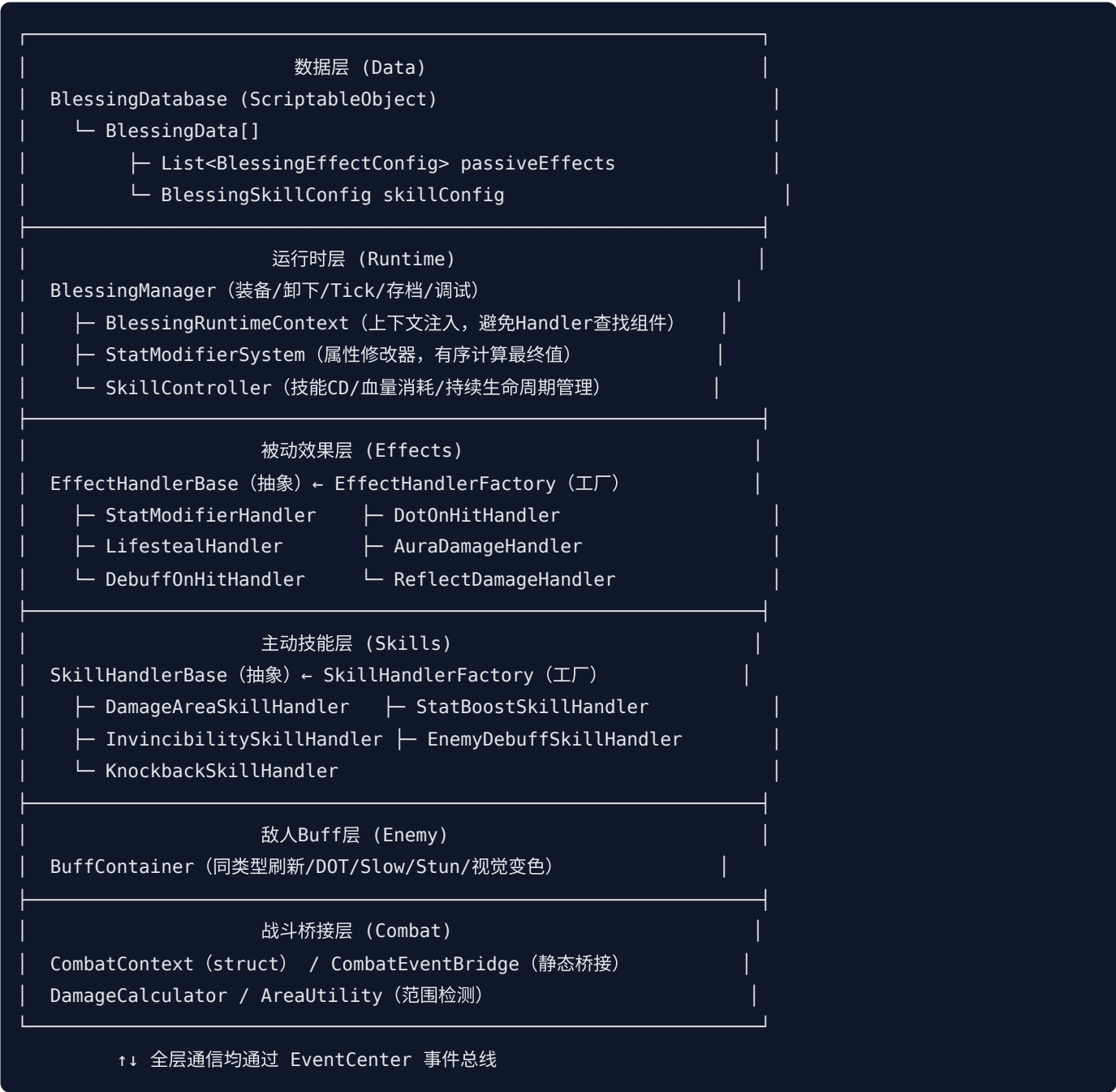
作者：费锦意

该系统模块应用于本人原创游戏《晦境孤剑：残玉为王》

Unity 2D · C# + ScriptableObject · 策略模式 · 工厂模式 · 事件驱动架构 · 属性修改器系统

一、整体架构概览

祝福系统是一个数据驱动 + 策略模式的框架。玩家同时只能装备一个祝福，每个祝福可包含多个被动效果与一个主动技能。祝福通过 NPC 购买，效果覆盖属性增强、攻击回血、命中施毒、范围光环、反伤、敌人减益等多种类型。



核心理念：

- **ScriptableObject 数据驱动** — 策划在 BlessingDatabase SO 中配置所有祝福，零代码
- **策略模式 + 工厂模式** — 被动效果与主动技能各自有抽象基类 + 工厂注册，新增类型零修改成本
- **属性修改器系统** — 统一管理所有属性加成，按固定顺序计算最终值，支持按来源批量移除
- **上下文注入模式** — BlessingRuntimeContext 传递给所有 Handler，避免每个 Handler 自行 GetComponent
- **事件驱动触发** — 被动效果按触发时机（命中/受击/击杀/Tick）注册到 EventCenter，战斗事件通过 CombatEventBridge 桥接

二、数据层：枚举定义与配置结构

2.1 BlessingEnums（全系统枚举定义）

文件： `Data/BlessingEnums.cs`

祝福系统的所有枚举集中定义于一个文件中，分为五大类：

枚举	用途	取值
EffectType	被动效果做什么	None / StatModifier / Lifesteal / DotOnHit / DebuffOnHit / AuraDamage / ReflectDamage
EffectTriggerType	被动效果何时生效	Permanent / OnAttackHit / OnPlayerHurt / OnEnemyKill / Tick
SkillType	主动技能类型	None / DamageArea / Invincibility / StatBoost / EnemyDebuff / Knockback
SkillDurationType	技能持续类型	Timed / OneShot / Toggle / ManualClose
HealthCostType	血量消耗类型	None / CurrentHPPct / MaxHPPct / Flat
DamageScaleType	伤害缩放基准	Flat / PlayerAttack1~3Pct / PlayerHeavyPct / EnemyMaxHPPct / EnemyCurHPPct
StatType	可修改属性	MaxHP / MoveSpeed / JumpForce / CritRate / CritMultiplier / AttackPower / HeavyAttackCD
ModifierOp	属性操作类型	AddFlat / AddPercentage / Multiply / Override
AreaShape	范围形状	Circle / Rectangle

设计要点：

- EffectType 与 EffectTriggerType 正交分离——"做什么"与"何时做"独立配置，策划可自由组合
- DamageScaleType 覆盖固定值、各段攻击力百分比、敌人血量百分比，适配多种伤害设计
- ModifierOp 四种操作覆盖加法、百分比、乘法、覆盖全部场景

2.2 BlessingEffectConfig & BlessingSkillConfig（配置结构体）

文件： `Data/BlessingConfigs.cs`

两个 [Serializable] 配置结构体，作为 BlessingData 的内嵌字段，在 Inspector 中直接配置。

BlessingEffectConfig（被动效果配置）

```

[Serializable]
public class BlessingEffectConfig
{
    public EffectType effectType;          // 效果类型
    public EffectTriggerType triggerType; // 触发时机

    // — 属性修改 (仅 StatModifier) —
    public StatType statType;
    public ModifierOp modifierOp;
    public float modifierValue;

    // — 伤害参数 (DotOnHit / AuraDamage / ReflectDamage) —
    public DamageScaleType damageScaleType;
    public float damageScale;           // 缩放比例, 0.2=20%
    public float duration;              // DOT/Debuff持续时间
    public float tickInterval;          // Tick间隔

    // — 范围参数 (仅 AuraDamage) —
    public AreaShape areaShape;
    public float areaRadius;
    public float areaWidth / areaHeight;
    public bool centerOnPlayer;
    public int maxTargets;              // 0=无限

    // — 回血参数 (仅 Lifesteal) —
    public float lifestealRatio;         // 占伤害比例
    public float lifestealMaxHPRatio;    // 占自身最大血量比例

    // — Debuff参数 (仅 DebuffOnHit) —
    public BuffType debuffType;
    public float slowRatio;              // 减速比例 0.5=50%
    public bool canAffectBoss;          // Boss是否免疫
}

```

设计要点：不同 EffectType 使用不同参数区域，未使用的参数在 Inspector 中忽略。策划只需选择 effectType + triggerType，填写对应区域的参数即可完成配置。

BlessingSkillConfig（主动技能配置）

```

[Serializable]
public class BlessingSkillConfig
{
    public SkillType skillType;
    public Sprite skillIcon;
    public SkillDurationType durationType;
    public float duration;           // 持续时间 (Timed型)
    public float cooldown;          // 冷却时间

    // 血量消耗
    public HealthCostType healthCostType;
    public float healthCostValue;

    // 技能伤害 + 范围 + 附带效果 (眩晕/击退)
    public DamageScaleType damageScaleType;
    public float damageScale;
    public AreaShape areaShape;
    public float areaRadius / areaWidth / areaHeight;
    public bool centerOnPlayer;
    public int maxTargets;
    public float stunDuration;
    public float knockbackForce;

    // 属性增强 (仅 StatBoost)
    public StatType boostStat;
    public ModifierOp boostOp;
    public float boostValue;

    // Debuff参数 (仅 EnemyDebuff)
    public BuffType debuffType;
    public float slowRatio;
    public bool canAffectBoss;
}

```

2.3 BlessingData (祝福数据定义)

文件: `Data/BlessingData.cs`

```
[Serializable]
public class BlessingData
{
    public string blessingID;           // 唯一标识, 存档用
    public string blessingName;
    public string description;
    public Sprite icon;
    public int cost;                    // 购买价格 (金币)

    public List<BlessingEffectConfig> passiveEffects = new();
    public BlessingSkillConfig skillConfig = new();

    public static BlessingData None { get; } = new() { blessingID = "None", ... };
}
```

一个祝福 = 多个被动效果 + 一个主动技能。 `BlessingData.None` 提供空祝福作为初始默认状态。

2.4 BlessingDatabase (ScriptableObject 数据库)

文件: `Data/BlessingDatabase.cs`

```
[CreateAssetMenu(fileName = "BlessingDatabase", menuName = "Blessing/BlessingDatabase")]
public class BlessingDatabase : ScriptableObject
{
    public List<BlessingData> blessings = new();

    public BlessingData FindByID(string id)
        => blessings.Find(b => b.blessingID == id);
}
```

策划通过右键 `Create > Blessing > BlessingDatabase` 创建数据库资产，在 Inspector 列表中添加所有祝福配置。
`FindByID` 供存档反序列化时按 ID 还原 `BlessingData` 引用。

三、运行时层：核心管理器与辅助系统

3.1 BlessingRuntimeContext (上下文注入)

文件: `Runtime/BlessingRuntimeContext.cs`

```
public class BlessingRuntimeContext
{
    public GameObject Player;
    public PlayerStat PlayerStat;
    public StatModifierSystem Modifiers;
    public BlessingDatabase Database;
    public Transform PlayerTransform;
}
```

纯数据容器，由 BlessingManager 在 Awake 时构造，传递给所有 EffectHandler 和 SkillHandler。避免每个 Handler 自行 GetComponent 查找依赖，降低耦合且减少性能开销。

3.2 StatModifierSystem（属性修改器系统）

文件：Runtime/StatModifierSystem.cs

修改器结构

```
public struct StatModifier
{
    public StatType Stat;
    public ModifierOp Op;
    public float Value;
    public object Source;    // 来源对象（卸下祝福时按来源批量移除）
}
```

使用 struct 避免 GC 开销。Source 字段记录修改器来源（通常为 BlessingData 或 SkillHandler 实例），卸下祝福时调用 RemoveAllFromSource 批量移除。

有序计算模型

```
GetFinalValue(stat, baseValue)
→ 1. AddFlat 累加：    result = baseValue + Σ(flat)
→ 2. AddPercentage 累加：result += baseValue × Σ(percentage)
→ 3. Multiply 叠乘：    result *= Π(multiplier)
→ 4. Override 覆盖：    result = 最后一个Override值
```

计算顺序固定为 AddFlat → AddPercentage → Multiply → Override。百分比基于基础值计算，乘法叠乘——符合常见 RPG 属性公式设计。

应用场景：PlayerStat.GetAttackDamage 中调用 Modifiers.GetFinalValue(StatType.AttackPower, 1f) 获取攻击力倍率，GetFinalValue(StatType.CritRate, Base_CritRate) 获取最终暴击率。

3.3 SkillController（主动技能控制器）

文件：Runtime/SkillController.cs

类型：普通 C# 类（非 MonoBehaviour），由 BlessingManager 创建并每帧驱动

职责

管理技能的冷却、血量消耗、持续生命周期，不直接读取输入（由 PlayerStateMachine 调用 TryActivate）。

技能释放流程

```

PlayerStateMachine.TryUseBlessing()
→ SkillController.TryActivate()
    └─ Toggle型 + 已激活 → DeactivateSkill(), 返回 false (不进入动画)
    └─ CD未冷却 → 返回 false
    └─ 血量不足 → 返回 false
    └─ 全部通过 → 返回 true
→ ChangeState(UseBlessing)
→ [动画播放完毕]
→ SkillController.OnSkillAnimationComplete()
    → handler.Activate(config, context) ← 实际执行技能效果
    → IsActive = true
    → _cooldownTimer = config.cooldown
    └─ OneShot型 → handler.Deactivate() + IsActive = false
    └─ Timed型 → _durationTimer = config.duration
    └─ Toggle/ManualClose → 保持激活, 等待关闭条件

```

血量消耗检查

```

float cost = config.healthCostType switch
{
    CurrentHPPct => stat.Curr_HP * config.healthCostValue,
    MaxHPPct     => stat.GetMaxHP() * config.healthCostValue,
    Flat         => config.healthCostValue,
    _ => 0f
};
if (stat.Curr_HP <= cost) return false; // 血量不足则无法释放
stat.TakeDamage(Mathf.RoundToInt(cost));

```

每帧更新（由 BlessingManager.Update 调用）

```

CustomUpdate()
→ CD倒计时递减
→ IsActive 时：
    └─ Timed型 → _durationTimer 递减 → 到期 DeactivateSkill()
    └─ Toggle/ManualClose → handler.Tick() 持续效果

```

3.4 BlessingManager（祝福核心管理器）

文件：Runtime/BlessingManager.cs

类型：MonoBehaviour，[RequireComponent(typeof(PlayerStat))], 挂载于玩家根节点

核心字段

```

public BlessingData CurrentBlessing { get; private set; }
public StatModifierSystem Modifiers { get; private set; }
public SkillController SkillController { get; private set; }

private readonly List<EffectHandlerBase> _activeHandlers = new();
private readonly List<(EventName, Action<CombatContext>)> _registeredListeners = new();
private readonly List<(EffectHandlerBase, BlessingEffectConfig, float)> _tickers = new();

```

三个列表分别追踪：当前激活的效果处理器、已注册的事件监听器（供卸下时反注册）、Tick型效果的计时器。

装备/卸下流程

```

EquipBlessing(blessing)
→ 1. UnequipCurrent()      ← 清理旧祝福
→ 2. CurrentBlessing = blessing
→ 3. SetupPassiveEffects(blessing)  ← 创建Handler + 注册事件/Tick
→ 4. SkillController.SetupSkill(blessing.skillConfig)
→ 5. SaveBlessing()        ← 写入存档

UnequipCurrent()
→ Modifiers.RemoveAllFromSource(CurrentBlessing)  ← 移除属性修改器
→ foreach handler.OnUnequip(context)              ← 通知Handler清理
→ _activeHandlers.Clear()
→ UnregisterAllListeners()                        ← 反注册事件
→ _tickers.Clear()                                ← 清除Tick
→ SkillController.DeactivateSkill() + ClearSkill()
→ CurrentBlessing = null

```

被动效果注册

```

private void SetupPassiveEffects(BlessingData blessing)
{
    foreach (var config in blessing.passiveEffects)
    {
        var handler = EffectHandlerFactory.Create(config.effectType);
        handler.OnEquip(config, _context);
        _activeHandlers.Add(handler);

        if (config.triggerType == EffectTriggerType.Tick)
            _tickers.Add((handler, config, config.tickInterval));
        else
            RegisterTrigger(config, handler); // 注册到 EventCenter
    }
}

```

事件触发映射


```

eventName eventName = config.triggerType switch
{
    OnAttackHit => eventName.EnemyHurt,
    OnPlayerHurt => eventName.PlayerHurt,
    OnEnemyKill => eventName.EnemyDie,
    _ => eventName.EnemyHurt
};

```

被动效果按触发时机注册到对应事件。例如"命中施加减速"注册到 `EnemyHurt` 事件, "反伤"注册到 `PlayerHurt` 事件。

Tick 效果驱动

```

UpdateTickEffects()
→ 遍历 _tickers, 每帧递减计时器
→ 计时器归零 → handler.OnTrigger(config, context, default)
→ 重置计时器 = config.tickInterval

```

光环伤害等 Tick 型效果由 `BlessingManager` 统一驱动, Handler 无需自行管理协程。

存档

```

SaveBlessing()
→ playerData.Current_Blessing = CurrentBlessing?.blessingID
→ stat.SaveData() ← 写入 player.json

LoadBlessingFromSave()
→ id = stat.playerData.Current_Blessing
→ database.FindByID(id) → EquipBlessing(blessing)

```

存档仅保存 `blessingID` 字符串, 读档时通过 `BlessingDatabase.FindByID` 还原引用。

调试切换

```

// F1 = 无祝福, F2~F9 = 数据库中第1~8个祝福
// 需在 Inspector 勾选 _enableDebugSwitch

```

四、被动效果层：策略模式 + 工厂模式

4.1 EffectHandlerBase (抽象基类)

文件: `Effects/EffectHandlerBase.cs`

```

public abstract class EffectHandlerBase
{
    public abstract void OnEquip(BlessingEffectConfig config, BlessingRuntimeContext context);
    public abstract void OnUnequip(BlessingRuntimeContext context);
    public abstract void OnTrigger(BlessingEffectConfig config, BlessingRuntimeContext context,
    CombatContext combat);
}

```

三个生命周期方法：

- `OnEquip` — 装备时调用（永久效果在此添加属性修改器）
- `OnUnequip` — 卸下时调用（清理资源）
- `OnTrigger` — 事件触发时调用（命中/受击/击杀/Tick）

4.2 EffectHandlerFactory（工厂注册）

```
private static readonly Dictionary<EffectType, Func<EffectHandlerBase>> _registry = new()
{
    { EffectType.StatModifier, () => new StatModifierHandler() },
    { EffectType.Lifesteal, () => new LifestealHandler() },
    { EffectType.DotOnHit, () => new DotOnHitHandler() },
    { EffectType.DebuffOnHit, () => new DebuffOnHitHandler() },
    { EffectType.AuraDamage, () => new AuraDamageHandler() },
    { EffectType.ReflectDamage, () => new ReflectDamageHandler() },
};

public static void Register(EffectType type, Func<EffectHandlerBase> factory)
    => _registry[type] = factory;
```

`EffectType` 枚举 → 处理器实例的映射表。新增效果类型时：1) 添加枚举值 2) 继承基类 3) 在工厂注册——不修改任何现有代码。

4.3 六种被动效果处理器

处理器	效果类型	触发时机	行为
<code>StatModifierHandler</code>	<code>StatModifier</code>	<code>Permanent</code>	<code>OnEquip</code> 时添加属性修改器到 <code>StatModifierSystem</code> ； <code>OnUnequip</code> 由 <code>BlessingManager</code> 统一移除
<code>LifestealHandler</code>	<code>Lifesteal</code>	<code>OnAttackHit</code>	按伤害比例和/或最大血量比例回血，调用 <code>PlayerStat.Heal()</code>
<code>DotOnHitHandler</code>	<code>DotOnHit</code>	<code>OnAttackHit</code>	给敌人 <code>BuffContainer</code> 添加 DOT Buff，伤害由 <code>DamageCalculator</code> 在 Tick 时实时计算
<code>DebuffOnHitHandler</code>	<code>DebuffOnHit</code>	<code>OnAttackHit</code>	给敌人施加 Slow/Stun 等 Debuff；Boss 免疫检查（ <code>isBoss</code> && <code>!canAffectBoss</code> 则跳过）
<code>AuraDamageHandler</code>	<code>AuraDamage</code>	<code>Tick</code>	每 <code>tickInterval</code> 检测范围内敌人并造成伤害，使用 <code>AreaUtility</code> 范围检测
<code>ReflectDamageHandler</code>	<code>ReflectDamage</code>	<code>OnPlayerHurt</code>	玩家受击时对攻击者造成伤害

StatModifierHandler 实现

```
public override void OnEquip(config, context)
{
    context.Modifiers.Add(config.statType, config.modifierOp, config.modifierValue, config);
    // 以config自身为Source标记，卸下时RemoveAllFromSource统一移除
}
```

DotOnHitHandler 实现

```
public override void OnTrigger(config, context, combat)
{
    var buffContainer = combat.Target.GetComponent<BuffContainer>();
    var dotBuff = new BuffInstance
    {
        buffType = BuffType.DOT,
        duration = config.duration,
        tickInterval = config.tickInterval,
        damageScaleType = config.damageScaleType,
        damageScale = config.damageScale,
        source = context.PlayerStat,
    };
    buffContainer.AddBuff(dotBuff);
}
```

DebuffOnHitHandler 的 Boss 免疫机制

```
var enemyStat = combat.Target.GetComponent<EnemyStatBase>();
if (enemyStat != null && enemyStat.isBoss && !config.canAffectBoss) return;
```

策划在配置中通过 `canAffectBoss` 布尔值控制每种 Debuff 是否对 Boss 生效——眩晕等强控默认对 Boss 免疫，减速等软控可配置为对 Boss 生效。

五、主动技能层：策略模式 + 工厂模式

5.1 SkillHandlerBase（抽象基类）

文件：Skills/SkillHandlerBase.cs

```
public abstract class SkillHandlerBase
{
    public abstract void Activate(BlessingSkillConfig config, BlessingRuntimeContext context);
    public abstract void Deactivate(BlessingRuntimeContext context);
    public virtual void Tick(BlessingSkillConfig config, BlessingRuntimeContext context) { }
}
```

5.2 SkillHandlerFactory（工厂注册）

```
private static readonly Dictionary<SkillType, Func<SkillHandlerBase>> _registry = new()
{
    { SkillType.DamageArea, () => new DamageAreaSkillHandler() },
    { SkillType.Invincibility, () => new InvincibilitySkillHandler() },
    { SkillType.StatBoost, () => new StatBoostSkillHandler() },
    { SkillType.EnemyDebuff, () => new EnemyDebuffSkillHandler() },
    { SkillType.Knockback, () => new KnockbackSkillHandler() },
};
```

与被动效果层完全对称的设计，新增技能类型同样遵循三步扩展法。

5.3 五种主动技能处理器

处理器	技能类型	行为
DamageAreaSkillHandler	DamageArea	范围伤害，可附带眩晕+击退；持续型每 Tick 重复造成伤害
InvincibilitySkillHandler	Invincibility	Activate 时 <code>PlayerStat.SetInvincible(true)</code> ，Deactivate 时恢复
StatBoostSkillHandler	StatBoost	Activate 时添加临时属性修改器（以 this 为 Source），Deactivate 时按来源移除
EnemyDebuffSkillHandler	EnemyDebuff	对范围内敌人施加配置类型 Debuff，含 Boss 免疫检查
KnockbackSkillHandler	Knockback	对范围内敌人施加击退力 + 眩晕

DamageAreaSkillHandler — 附带效果组合

```
private void ApplyDamage(config, context)
{
    var targets = AreaUtility.GetTargetsInArea(...);
    foreach (var target in targets)
    {
        enemyStat.CurHP -= DamageCalculator.CalculateDamage(...);

        // 附带眩晕
        if (config.stunDuration > 0f)
            buff?.AddBuff(new BuffInstance { buffType = BuffType.Stun, duration =
config.stunDuration });

        // 附带击退
        if (config.knockbackForce > 0f)
            rb.AddForce(dir * config.knockbackForce, ForceMode2D.Impulse);
    }
}
```

一个技能可同时造成伤害 + 眩晕 + 击退，通过 config 中的参数开关控制，策划可自由组合。

StatBoostSkillHandler — 临时属性增强

```
public override void Activate(config, context)
=> context.Modifiers.Add(config.boostStat, config.boostOp, config.boostValue, this);

public override void Deactivate(context)
=> context.Modifiers.RemoveAllFromSource(this);
```

技能激活时添加属性修改器（如暴击率+50%），技能结束时按来源 this 批量移除——复用 StatModifierSystem 统一管理，无需额外属性管理逻辑。

六、敌人 Buff 层：BuffContainer

文件：Enemy/BuffContainer.cs

类型： `MonoBehaviour`，`[RequireComponent(typeof(EnemyStatBase))]`，挂载于敌人

6.1 BuffType 与 BuffInstance

```
public enum BuffType
{
    DOT,           // 持续伤害
    Stun,          // 眩晕
    StatReduction, // 属性削弱
    Slow,          // 减速
}

public class BuffInstance
{
    public BuffType buffType;
    public float duration;
    public float tickInterval;
    public float tickTimer;
    public DamageScaleType damageScaleType; // DOT伤害缩放
    public float damageScale;
    public object source;                  // 来源 (PlayerStat, 用于获取攻击力)
    public float slowRatio = 0.5f;        // 减速比例
}
```

6.2 核心机制

同类型刷新（不叠加）

```
public void AddBuff(BuffInstance buff)
{
    for (int i = 0; i < _buffs.Count; i++)
    {
        if (_buffs[i].buffType == buff.buffType)
        {
            _buffs[i].duration = buff.duration; // 刷新时长
            if (buff.buffType == BuffType.Slow)
                _buffs[i].slowRatio = buff.slowRatio; // 更新减速比例
            UpdateVisualColor();
            return;
        }
    }
    buff.tickTimer = buff.tickInterval;
    _buffs.Add(buff);
    UpdateVisualColor();
}
```

同类型 Buff 重复施加时刷新持续时间而非叠加，防止减速/眩晕无限叠加导致敌人完全失效。

Tick 与到期清理

```
Update()
→ 遍历 _buffs (倒序, 安全移除)
→ duration 递减
→ tickInterval > 0 时 tickTimer 递减 → ApplyTick
→ duration <= 0 时 RemoveAt
→ 列表变化时 UpdateVisualColor
```

DOT Tick 伤害 + 事件转发

```
case BuffType.DOT:
    int damage = DamageCalculator.CalculateDamage(
        buff.damageScaleType, buff.damageScale,
        buff.source as PlayerStat, _stat);
    _stat.CurHP -= damage;
    EventCenter.Instance.Invoke(EventName.EnemyHurt, new CombatContext { ... });
```

DOT Tick 造成的伤害也会触发 `EnemyHurt` 事件, 使攻击回血等祝福效果能响应 DOT 伤害。

视觉反馈

```
// 优先级: Stun (深蓝) > Slow (浅蓝半透明) > 原色
if (HasBuff(BuffType.Stun))
    _spriteRenderer.color = StunColor;
else if (HasBuff(BuffType.Slow))
    _spriteRenderer.color = SlowColor;
else
    _spriteRenderer.color = _originalColor;
```

Buff 状态变化时自动更新敌人 `SpriteRenderer` 颜色, 提供即时视觉反馈。

对外查询接口

```
public bool HasBuff(BuffType type);        // 敌人AI查询是否眩晕
public float GetSlowRatio();               // 返回最高减速比例
public void ClearAll();                    // 清除所有Buff
```

`HasBuff(BuffType.Stun)` 供 `EnemyAlMelee.CheckStun()` 调用, 眩晕时冻结 AI 行为; `GetSlowRatio()` 供 `EnemyStatBase` 计算实际巡逻/追击速度。

七、战斗桥接层：事件转发与伤害计算

7.1 CombatContext (战斗上下文)

文件: `Combat/CombatContext.cs`

```
public struct CombatContext
{
    public GameObject Attacker;
    public GameObject Target;
    public int Damage;
    public bool IsCrit;
    public int ComboStage;    // 0=重击, 1-3=连击段
}
```

使用 struct 避免每次战斗交互的 GC 分配。作为 `EnemyHurt` / `PlayerHurt` / `EnemyDie` 事件的参数传递。

7.2 CombatEventBridge（静态桥接器）

文件： `Combat/CombatEventBridge.cs`

```
public static class CombatEventBridge
{
    public static void OnPlayerAttackHit(player, enemy, damage, isCrit, comboStage)
        => EventCenter.Invoke(EventName.EnemyHurt, new CombatContext { Attacker = player, Target = enemy, ... });

    public static void OnEnemyAttackHit(enemy, player, damage, isCrit)
        => EventCenter.Invoke(EventName.PlayerHurt, new CombatContext { ... });

    public static void OnEnemyDie(player, enemy)
        => EventCenter.Invoke(EventName.EnemyDie, new CombatContext { ... });

    public static void OnPlayerDie(enemy, player)
        => EventCenter.Invoke(EventName.PlayerDie, new CombatContext { ... });
}
```

将战斗系统的原生回调（`AttackHitbox.OnHit`、`EnemyAlMelee.TakeDamage` 等）转换为 `EventCenter` 全局事件。战斗模块只需调用静态方法，无需感知祝福系统的存在——**单向依赖，战斗不引用祝福**。

7.3 DamageCalculator（伤害计算工具）

文件： `Combat/DamageCalculator.cs`

```

public static int CalculateDamage(DamageScaleType scaleType, float scale,
    PlayerStat playerStat, EnemyStatBase enemyStat)
{
    return scaleType switch
    {
        Flat => Mathf.RoundToInt(scale),
        PlayerAttack1Pct => Mathf.RoundToInt(playerStat.GetBaseAttackPower(1) * scale),
        PlayerAttack2Pct => Mathf.RoundToInt(playerStat.GetBaseAttackPower(2) * scale),
        PlayerAttack3Pct => Mathf.RoundToInt(playerStat.GetBaseAttackPower(3) * scale),
        PlayerHeavyPct => Mathf.RoundToInt(playerStat.GetBaseAttackPower(0) * scale),
        EnemyMaxHPPct => Mathf.RoundToInt(enemyStat.MaxHP * scale),
        EnemyCurHPPct => Mathf.RoundToInt(enemyStat.CurHP * scale),
    };
}

```

根据 `DamageScaleType` 从不同基准计算伤害——固定值、各段攻击力百分比、敌人血量百分比。供 `DotOnHitHandler`、`AuraDamageHandler`、`DamageAreaSkillHandler` 等统一调用。

7.4 AreaUtility（范围检测工具）

```

public static List<GameObject> GetTargetsInArea(Vector3 center, AreaShape shape,
    float radius, float width, float height, int maxTargets)
{
    // 圆形 → Physics2D.OverlapCircleAll
    // 矩形 → Physics2D.OverlapAreaAll
    // 过滤：通过 EnemyStatBase 组件判断是否为敌人
    // 限量：按距离排序后取最近的 maxTargets 个
}

```

统一范围检测逻辑，支持圆形/矩形两种形状 + 最大目标数限制。供 `AuraDamageHandler`、`DamageAreaSkillHandler`、`EnemyDebuffSkillHandler`、`KnockbackSkillHandler` 共享调用。

八、PlayerStat 与祝福系统的集成

8.1 伤害公式集成


```

public int GetAttackDamage(int comboStage, out bool isCrit)
{
    int baseAP = GetBaseAttackPower(comboStage);

    var bm = GetComponent<BlessingManager>();
    float apMultiplier = bm?.Modifiers.GetFinalValue(StatType.AttackPower, 1f) ?? 1f;
    float critRate      = bm?.Modifiers.GetFinalValue(StatType.CritRate, Base_CritRate) ??
Base_CritRate;
    float critMulti     = bm?.Modifiers.GetFinalValue(StatType.CritMultiplier, Base_CritMulti) ??
Base_CritMulti;

    int currAP = Mathf.RoundToInt(baseAP * apMultiplier);
    isCrit = Random.value < critRate;
    return isCrit ? Mathf.RoundToInt(currAP * critMulti) : currAP;
}

```

PlayerStat 通过 `BlessingManager.Modifiers.GetFinalValue` 获取祝福加成后的最终属性值。`??` 运算符确保无 BlessingManager 时回退到基础值，兼容性安全。

8.2 无敌状态

```

private bool _isInvincible = false;

public void SetInvincible(bool value) => _isInvincible = value;

public void TakeDamage(int damage)
{
    if (_isInvincible) return; // 无敌时直接跳过
    // ... 额外生命优先扣除 → HP 扣除
}

```

`SetInvincible` 由 `InvincibilitySkillHandler` 调用，`TakeDamage` 入口处检查 `_isInvincible` 直接跳过伤害。

8.3 存档集成

```

PlayerData.Current_Blessing ← string, 存储当前祝福ID
SaveData() → JsonUtility.ToJson → player.json
LoadData() → JsonUtility.FromJson → BlessingManager.LoadBlessingFromSave()

```

祝福ID作为字符串存入 `player.json`，读档时通过 `BlessingDatabase.FindByID` 还原。

九、事件总线：EventCenter 通信映射

事件名	发布方	订阅方	携带参数	用途

EnemyHurt	CombatEventBridge.OnPlayerAttackHit	BlessingManager (Lifesteal/DotOnHit/DebuffOnHit Handler)	CombatContext	玩 家 命 中 敌 人 → 触 发 命 中 型 被 动 效 果
PlayerHurt	CombatEventBridge.OnEnemyAttackHit	BlessingManager (ReflectDamage Handler)	CombatContext	玩 家 受 击 → 触 发 反 伤 等 被 动 效 果
EnemyDie	CombatEventBridge.OnEnemyDie	BlessingManager (OnEnemyKill 型Handler)	CombatContext	击 杀 敌 人 → 触 发 击 杀 型 被 动 效 果
PlayerDie	CombatEventBridge.OnPlayerDie	(预留)	CombatContext	玩 家 死 亡 事 件

BlessingChanged	(预留, 当前未调用)	—	无	预留供UI监听祝福切换
SkillActivated	(预留, 当前未调用)	—	无	预留供UI监听技能激活
SkillDeactivated	(预留, 当前未调用)	—	无	预留供UI监听技能结束

十、战斗交互全链路

玩家攻击命中 → 触发被动效果

```
玩家按 J 键 → AttackHitbox.Activate()  
  → DetectionZone.OnTargetEnter → AttackHitbox.OnHit.Invoke(target)  
  → PlayerStateMachine.OnPlayerAttackHit(target)  
  → 计算伤害 (GetAttackDamage, 含祝福属性加成)  
  → CombatEventBridge.OnPlayerAttackHit(player, enemy, damage, isCrit, comboStage)  
  → EventCenter.Invoke(EnemyHurt, CombatContext)  
  → BlessingManager 注册的监听器触发:  
    └─ LifestealHandler.OnTrigger() → PlayerStat.Heal(damage × ratio)  
    └─ DotOnHitHandler.OnTrigger() → BuffContainer.AddBuff(DOT)  
    └─ DebuffOnHitHandler.OnTrigger() → BuffContainer.AddBuff(Slow/Stun)  
  → enemyAI.TakeDamage(damage)
```

玩家受击 → 触发反伤

```
EnemyAIMelee 攻击命中玩家
→ CombatEventBridge.OnEnemyAttackHit(enemy, player, damage, isCrit)
→ EventCenter.Invoke(PlayerHurt, CombatContext)
→ ReflectDamageHandler.OnTrigger()
    → DamageCalculator.CalculateDamage(...)
    → enemyStat.CurHP -= damage
```

玩家释放主动技能

```
玩家按 L 键
→ PlayerController: blessingInput = true
→ PlayerStateMachine.TryUseBlessing()
→ SkillController.TryActivate() [CD/血量检查]
→ ChangeState(UseBlessing)
→ [动画播放完毕]
→ SkillController.OnSkillAnimationComplete()
→ SkillHandlerBase.Activate(config, context)
    ├── DamageAreaSkillHandler → AreaUtility + DamageCalculator + 附带眩晕/击退
    ├── InvincibilitySkillHandler → PlayerStat.SetInvincible(true)
    ├── StatBoostSkillHandler → Modifiers.Add(boostStat, this)
    ├── EnemyDebuffSkillHandler → BuffContainer.AddBuff(debuff)
    └── KnockbackSkillHandler → Rigidbody2D.AddForce + BuffContainer.AddBuff(Stun)
```

十一、策划配置指南

新增一个祝福的完整流程

1. 在 Unity 编辑器中选中 BlessingDatabase 资产
2. 在 Inspector 的 blessings 列表中点击 + 添加一条
3. 填写基础信息：blessingID（唯一）、blessingName、description、icon、cost
4. 配置被动效果：在 passiveEffects 列表中添加条目
 - 选择 effectType + triggerType
 - 填写对应参数区域（属性修改/伤害参数/范围参数/回血参数/Debuff参数）
5. 配置主动技能：在 skillConfig 中选择 skillType
 - 填写技能参数（CD/持续时间/血量消耗/伤害/范围/附带效果）
6. 完成，无需编写任何代码

配置示例：霜冻遗孤

配置项	值
blessingID	"1"
blessingName	霜冻遗孤
cost	500
被动效果	DebuffOnHit / OnAttackHit

debuffType	Slow
slowRatio	0.5（减速50%）
duration	3s
canAffectBoss	true
主动技能	Knockback
durationType	Timed
duration	5s
cooldown	45s
areaRadius	10m（圆形）
maxTargets	5
knockbackForce	配置值
stunDuration	5s

十二、设计模式总结

模式	应用位置	作用
策略模式	EffectHandlerBase / SkillHandlerBase	每种效果/技能为独立子类，新增类型不改旧代码
工厂模式	EffectHandlerFactory / SkillHandlerFactory	枚举→处理器映射表，集中注册，支持运行时扩展
观察者模式	EventCenter + CombatEventBridge	战斗事件触发被动效果，战斗模块不引用祝福系统
数据驱动模式	BlessingDatabase (ScriptableObject)	策划零代码配置祝福，数据与逻辑彻底分离
属性修改器系统	StatModifierSystem	统一管理所有属性加成，有序计算，按来源批量移除
上下文注入模式	BlessingRuntimeContext	依赖注入避免 Handler 自行查找组件，降低耦合
桥接模式	CombatEventBridge	将战斗系统原生回调转换为全局事件，隔离两个子系统
同类型刷新策略	BuffContainer.AddBuff	同类型 Buff 刷新时长而非叠加，防止无限叠加

感谢您的浏览！